

Temas Tratados en el Trabajo Práctico 8

- Aprendizaje estadístico.
- Evolución de la verosimilitud de una hipótesis en función de observaciones.
- Aprendizaje no supervisado. Algoritmo K-means.
- Aprendizaje supervisado. Algoritmo Knn.
- Aprendizaje por refuerzo. Algoritmo Q-Learning.
- Ejercicios Teóricos

1. Un fabricante de tornillos vende cajas que contienen 1000 tornillos de cabeza redonda con tres tipos de recubrimiento electrolítico (cincado, cobre y níquel). Cada caja se rellena con diferentes proporciones que pueden variar de la siguiente manera:

a: Todos los tornillos están recubiertos de níquel. 15 de cada 100 cajas se llenan de esta manera.

b: El 70% de los tornillos están recubiertos de níquel, el 20% de cobre, el resto está cincado. 15 de cada 100 cajas se llenan de esta manera.

c: El 50% de los tornillos están recubiertos de níquel, el 25% de cobre y el resto está cincado. 50 de cada 100 cajas se llenan de esta manera.

d: El 20 % de los tornillos están recubiertos de níquel, el 50% de cobre y el resto está cincado. 10 de cada 100 cajas se llenan de esta manera.

e: Todos los tornillos están recubiertos de cobre. 10 de cada 100 cajas se llenan de esta manera.

1.1 ¿Cuál es la distribución a priori sobre las hipótesis?

La distribución a priori refleja la probabilidad de que una caja haya sido llenada con una determinada proporción de recubrimientos, antes de observar ningún tornillo.

Se define así:

$h1$: Todos níquel $\rightarrow P(h1)=0,15$

$h2$: 70% níquel, 20% cobre, 10% cincado $\rightarrow P(h2)=0,15$

$h3$: 50% níquel, 25% cobre, 25% cincado $\rightarrow P(h3)=0,50$

$h4$: 20% níquel, 50% cobre, 30% cincado $\rightarrow P(h4)=0,10$

$h5$: Todos cobre $\rightarrow P(h5)=0,10$

Entonces, la distribución a priori es:

$P(H)=(0,15, 0,15, 0,50, 0,10, 0,10)$

Considerando que los 10 primeros tornillos que se extraen de una caja de muestra son de cobre:

1.2 Calcule la probabilidad de cada hipótesis dado que los 10 primeros tornillos fueron de cobre.

Para resolver este ejercicio utilizamos el **Teorema de Bayes**, que permite actualizar las probabilidades de nuestras hipótesis luego de observar nueva evidencia.

En este caso, las hipótesis (h_1) a (h_5) describen diferentes proporciones de recubrimiento de tornillos, y la evidencia es que los **10 primeros tornillos extraídos fueron de cobre**.

La fórmula de Bayes es:

$$P(h_i | E) = \frac{P(E | h_i) \cdot P(h_i)}{\sum_{j=1}^5 P(E | h_j) \cdot P(h_j)}$$

- Paso 1: Probabilidades a priori

Según el enunciado, las probabilidades a priori de cada hipótesis son:

$$P(h_1) = 0.15$$

$$P(h_2) = 0.15$$

$$P(h_3) = 0.50$$

$$P(h_4) = 0.10$$

$$P(h_5) = 0.10$$

- Paso 2: Probabilidades de la evidencia

Calculamos la probabilidad de obtener 10 tornillos de cobre según cada hipótesis. Esto equivale a elevar al exponente 10 la proporción de cobre en cada hipótesis:

$$P(E | h_1) = 0^{10} = 0$$

$$P(E | h_2) = 0.20^{10} \approx 1.024 \times 10^{-7}$$

$$P(E | h_3) = 0.25^{10} \approx 9.537 \times 10^{-7}$$

$$P(E | h_4) = 0.50^{10} \approx 9.766 \times 10^{-4}$$

$$P(E | h_5) = 1.00^{10} = 1$$

- Paso 3: Producto con la priori

$$P(E | h_1) \cdot P(h_1) = 0 \cdot 0.15 = 0$$

$$P(E | h_2) \cdot P(h_2) = 1.024 \times 10^{-7} \cdot 0.15 = 1.536 \times 10^{-8}$$

$$P(E | h_3) \cdot P(h_3) = 9.537 \times 10^{-7} \cdot 0.50 = 4.768 \times 10^{-7}$$

$$P(E | h_4) \cdot P(h_4) = 9.766 \times 10^{-4} \cdot 0.10 = 9.766 \times 10^{-5}$$

$$P(E | h_5) \cdot P(h_5) = 1 \cdot 0.10 = 0.10$$

- Paso 4: Probabilidad total de la evidencia

$$\begin{aligned} P(E) &= \sum_{i=1}^5 P(E | h_i) \cdot P(h_i) \approx 0 + 1.536 \times 10^{-8} + 4.768 \times 10^{-7} + 9.766 \times 10^{-5} + 0.10 \\ &= 0.10009766 \end{aligned}$$

- Paso 5: Cálculo de probabilidades a posteriori

$$P(h_1 | E) = \frac{0}{0.10009766} = 0$$

$$P(h_2 | E) = \frac{1.536 \times 10^{-8}}{0.10009766} \approx 1.53 \times 10^{-7}$$

$$P(h_3 | E) = \frac{4.768 \times 10^{-7}}{0.10009766} \approx 4.76 \times 10^{-6}$$

$$P(h_4 | E) = \frac{9.766 \times 10^{-5}}{0.10009766} \approx 9.76 \times 10^{-4}$$

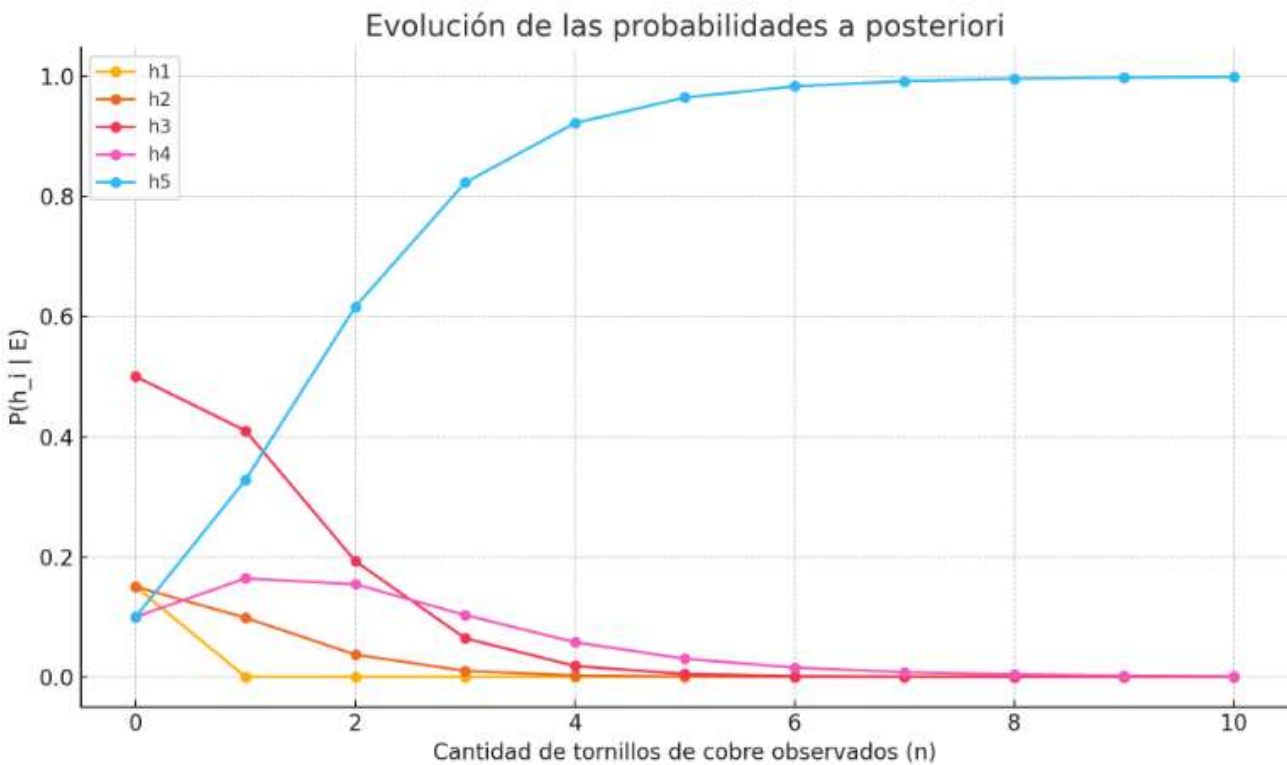
$$P(h_5 | E) = \frac{0.10}{0.10009766} \approx 0.9990$$

- Conclusión

Luego de observar que los 10 primeros tornillos extraídos fueron de cobre, la probabilidad más alta corresponde a la hipótesis (h_5) (todos los tornillos son de cobre), lo cual es coherente con la evidencia observada.

1.3 Grafique la evolución de la verosimilitud de cada hipótesis en función del número de tornillos extraídos de la caja.

n	$P(h_1 E)$	$P(h_2 E)$	$P(h_3 E)$	$P(h_4 E)$	$P(h_5 E)$
0	0.150000	0.150000	0.500000	0.100000	0.100000
1	0.000000	0.098361	0.409836	0.163934	0.327869
2	0.000000	0.036980	0.192604	0.154083	0.616333
3	0.000000	0.009876	0.064294	0.102870	0.822961
4	0.000000	0.002213	0.018011	0.057634	0.922142
5	0.000000	0.000443	0.004597	0.028729	0.966231
6	0.000000	0.000087	0.001133	0.013996	0.984784
7	0.000000	0.000017	0.000279	0.006787	0.992917
8	0.000000	0.000003	0.000068	0.003287	0.996642
9	0.000000	0.000001	0.000017	0.001586	0.998396
10	0.000000	0.000000	0.000004	0.000765	0.999231



1.4 Para cada hipótesis, ¿cuál es la probabilidad de que el cuarto tornillo extraído sea de cobre?

Análisis: Probabilidad de extraer 4 tornillos de cobre consecutivos sin reposición

Dado que las extracciones se realizan sin reposición, la probabilidad de extraer 4 tornillos de cobre seguidos desde una caja con (c) tornillos de cobre y (N = 1000) tornillos en total se calcula como:

$$P = \frac{c}{N} \cdot \frac{c-1}{N-1} \cdot \frac{c-2}{N-2} \cdot \frac{c-3}{N-3}$$

A continuación se presentan los resultados para cada hipótesis:

Hipótesis	%Cobre	Cantidad de tornillos de cobre	P(4 cobres seguidos sin reposición)
h_1	0%	0	0.00000000
h_2	20%	200	0.00156179
h_3	25%	250	0.00383616
h_4	50%	500	0.06212481
h_5	100%	1000	1.00000000

Conclusión:

La probabilidad de obtener 4 cobres consecutivos sin reposición varía significativamente entre hipótesis, siendo 0 en (h_1), 1 en (h_5), y aumentando progresivamente con la proporción de cobre en la caja.

2. ¿Qué diferencia principal existe entre un algoritmo supervisado y un algoritmo no supervisado?

Diferencia principal entre algoritmo supervisado y no supervisado

- Algoritmo supervisado: Aprende a partir de datos que incluyen etiquetas o salidas conocidas. Se le entrena con ejemplos donde se conoce cuál es la respuesta correcta, y luego puede predecir la salida para nuevos datos. Ejemplo: el algoritmo KNN (k-vecinos más cercanos), donde se le da al modelo un conjunto de datos con clases conocidas, y luego clasifica un nuevo ejemplo según esas clases.
- Algoritmo no supervisado: Aprende a partir de datos sin etiquetas, buscando patrones o estructuras ocultas dentro de los datos. No sabe de antemano qué está buscando, simplemente agrupa o estructura los datos según similitudes. Ejemplo: el algoritmo K-means, que agrupa los datos en clusters sin saber qué clase representa cada grupo.

- Ejercicios de implementación

3. Genere un conjunto de 23 puntos que contengan coordenadas xy aleatorias con valores contenidos en el intervalo [0, 5] y grafique los puntos obtenidos en un gráfico.

3.1 Implemente un algoritmo K-means que clasifique 20 de los puntos en 2 grupos y grafique el resultado asignando un color a cada uno.

3.2 Tome los tres puntos restantes y clasifíquelos en los grupos obtenidos anteriormente usando el algoritmo Knn. Utilice distintos valores de K y anote lo que observa con esta elección.

```
In [3]: import numpy as np
import matplotlib.pyplot as plt
```

```

from sklearn.cluster import KMeans
from sklearn.neighbors import KNeighborsClassifier

semilla = 26541321

# 1. Generar 23 puntos aleatorios (x, y) en [0, 5]
np.random.seed(semilla) # Semilla fija para reproducibilidad
puntos = np.random.uniform(0, 5, (23, 2))

# Dividir en entrenamiento (20 puntos) y prueba (3 puntos)
puntos_train = puntos[:20]
puntos_test = puntos[20:]

# 2. Graficar los 23 puntos originales y marcar con X los no usados
plt.figure(figsize=(6,6))
plt.scatter(puntos[:,0], puntos[:,1], c='black', marker='o', label="Todos los puntos")
plt.scatter(puntos_test[:,0], puntos_test[:,1], c='red', marker='x', s=100, label="No usados")
plt.title("23 puntos aleatorios en [0,5] (antes de clasificar)")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.grid(True)
plt.show()

# 3. K-means con 2 clusters en los primeros 20 puntos
kmeans = KMeans(n_clusters=2, random_state=semilla)
kmeans.fit(puntos_train)

labels_train = kmeans.labels_
centros = kmeans.cluster_centers_

# Graficar K-means con 20 puntos
plt.figure(figsize=(6,6))
plt.scatter(puntos_train[:,0], puntos_train[:,1], c=labels_train, cmap='coolwarm', marker='o')
plt.scatter(centros[:,0], centros[:,1], c='yellow', marker='X', s=200, label="Centros K-means")
plt.title("K-means (20 puntos iniciales)")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.grid(True)
plt.show()

# 4. Entrenar un clasificador KNN usando los labels de K-means
knn = KNeighborsClassifier(n_neighbors=3) # <--- Valor de k!
knn.fit(puntos_train, labels_train)

# Predecir los 3 puntos restantes
labels_test = knn.predict(puntos_test)

# Graficar clasificación final (23 puntos: train + test)
plt.figure(figsize=(6,6))
plt.scatter(puntos_train[:,0], puntos_train[:,1], c=labels_train, cmap='coolwarm', marker='o')
plt.scatter(puntos_test[:,0], puntos_test[:,1], c=labels_test, cmap='coolwarm', marker='^', s=100)
plt.scatter(centros[:,0], centros[:,1], c='yellow', marker='X', s=200, label="Centros K-means")
plt.title("Clasificación final (K-means + KNN)")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.grid(True)
plt.show()

# 5. Resumen: mostrar los 23 puntos clasificados
# Unir labels_train y labels_test para graficar todo junto

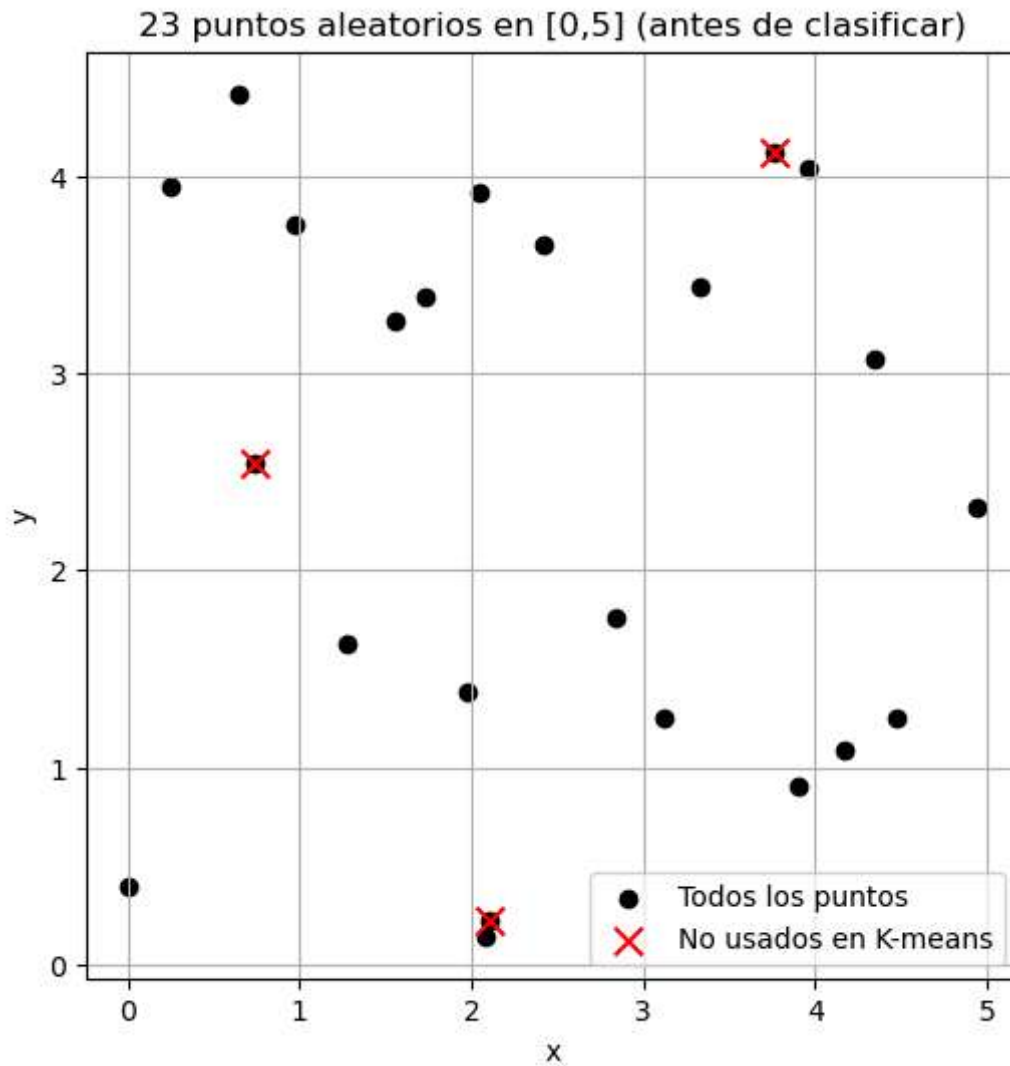
```

```

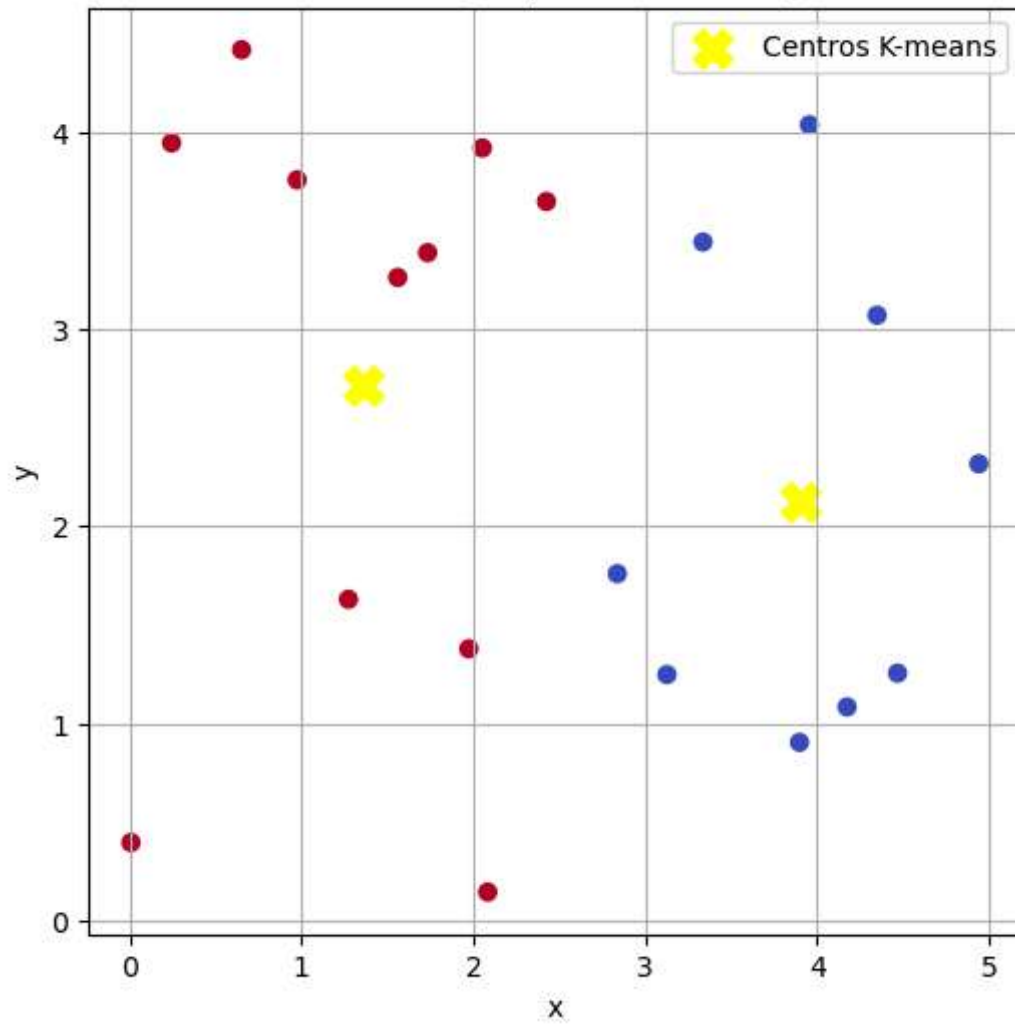
labels_final = np.concatenate([labels_train, labels_test])

plt.figure(figsize=(6,6))
plt.scatter(puntos[:,0], puntos[:,1], c=labels_final, cmap='coolwarm', marker='o')
plt.scatter(centros[:,0], centros[:,1], c='yellow', marker='X', s=200, label="Centros K-means")
plt.title("23 puntos clasificados (resumen)")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.grid(True)
plt.show()

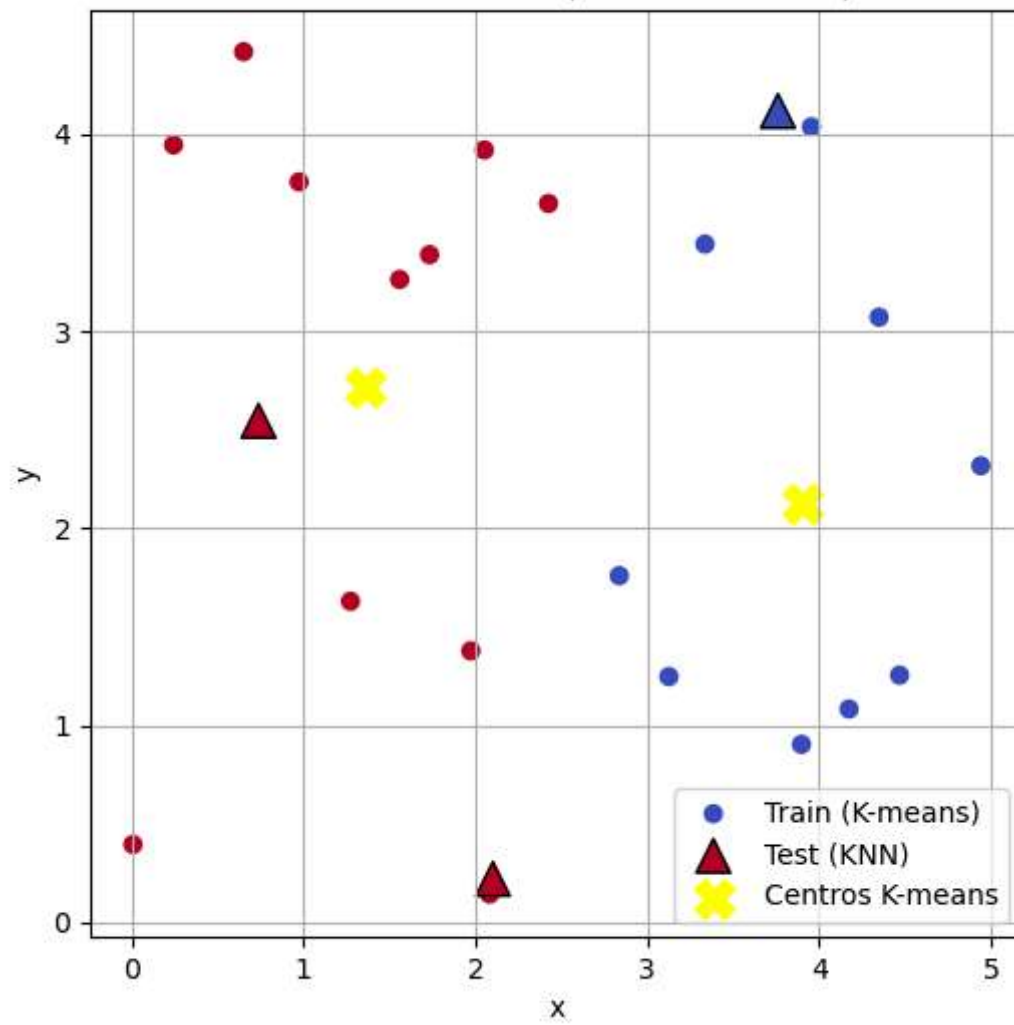
```

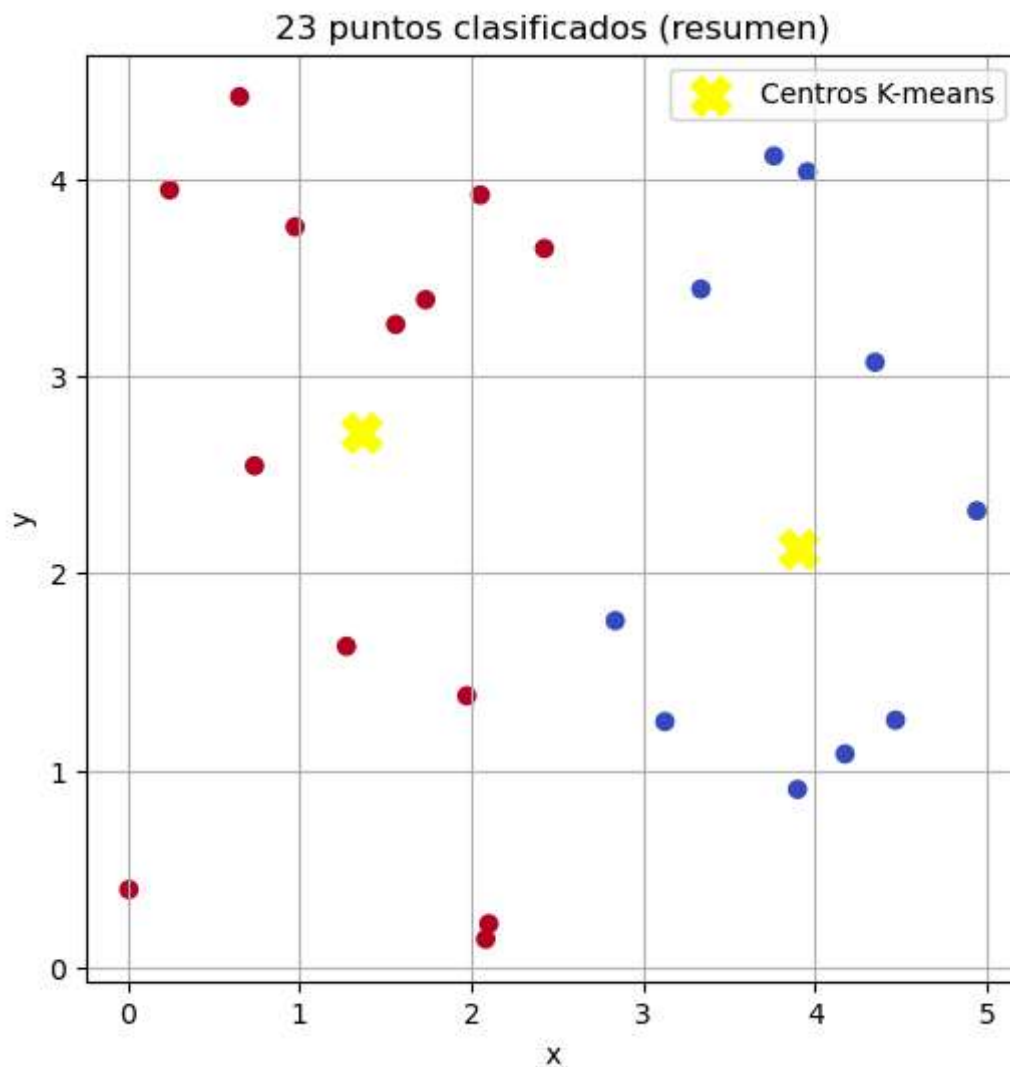


K-means (20 puntos iniciales)



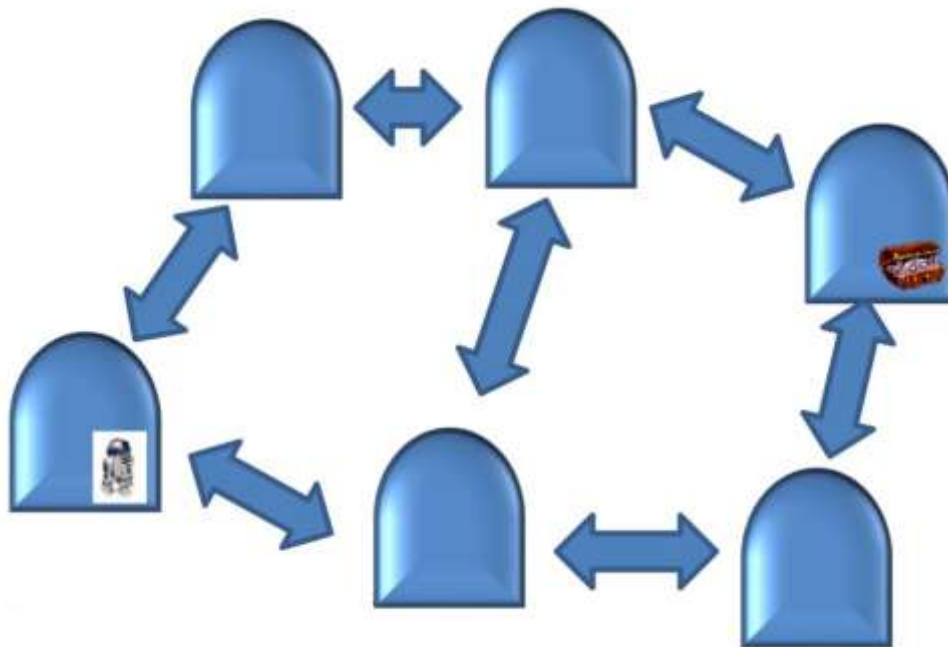
Clasificación final (K-means + KNN)



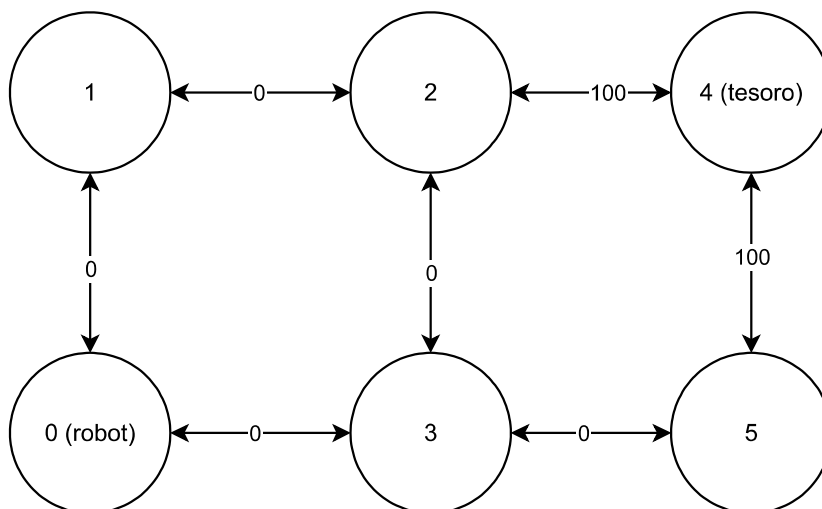


Modificando los valores de "K" se pueden llegar a modificar la clasificación de los puntos tomados en el algoritmo Knn, pero con un valor de $K=3$ se observó una clasificación precisa.

- La imagen mostrada abajo muestra una red de salas y cómo se comunican entre ellas. Implemente un algoritmo Q-Learning con un factor despreciativo $\gamma = 0.9$. La matriz de recompensas debe asignar un valor de 0 a cada camino accesible y un valor de 100 a caminos que lleven a la sala del tesoro.



4.1 Dibuje un diagrama con la asignación de recompensas correspondiente a cada estado.



4.2 Diseñe y muestre la matriz de recompensas.

```
In [4]: R = [
# 0  1  2  3  4  5
[ -1,  0, -1,  0, -1, -1 ], # 0
[  0, -1,  0, -1, -1, -1 ], # 1
[ -1,  0, -1,  0,100, -1 ], # 2
[  0, -1,  0, -1, -1,  0 ], # 3
[ -1, -1,  0, -1, -1,  0 ], # 4
[ -1, -1, -1,  0,100, -1 ], # 5
]
```

4.3 Obtenga la matriz Q óptima, normalícela respecto al valor máximo encontrado y grafique la política obtenida en el diagrama mostrado inicialmente.

```
In [5]: # Librerías necesarias
import numpy as np                # Para trabajar con matrices y operaciones numéricas
import matplotlib.pyplot as plt   # Para graficar la política óptima
import networkx as nx             # Para construir y visualizar grafos (salas conectadas)
import random                     # Para elegir acciones aleatorias durante el aprendizaje
from sklearn.preprocessing import MinMaxScaler # Para normalizar la matriz Q entre 0 y 1
```

```

import pandas as pd # Para mostrar resultados en forma de tabla

# Cada fila representa un estado (sala) y cada columna una posible acción (ir a otra sala)
# -1 significa que no se puede ir de un estado a otro
# 0 es una transición válida pero sin recompensa
# 100 es la recompensa por llegar al estado del TESORO

R = np.array([
    [-1, 0, -1, 0, -1, -1], # 0
    [ 0, -1, 0, -1, -1, -1], # 1
    [-1, 0, -1, 0, 100, -1], # 2
    [ 0, -1, 0, -1, -1, 0], # 3
    [-1, -1, 0, -1, 100, 0], # 4
    [-1, -1, -1, 0, 100, -1], # 5
])

# Parámetros
gamma = 0.9 # Factor de descuento: valora más las recompensas cercanas en el tiempo
alpha = 0.8 # Tasa de aprendizaje: cuánto se actualiza la Q cada vez
n_states = R.shape[0] # Cantidad de estados (6 en este caso)

# Inicializamos Q
Q = np.zeros_like(R, dtype=float) # Matriz Q, del mismo tamaño que R, inicializada en ceros

# Q-Learning
episodes = 10000 # Cantidad de episodios (iteraciones de entrenamiento)

for _ in range(episodes):
    state = random.randint(0, n_states - 1) # Elegimos un estado inicial al azar
    possible_actions = np.where(R[state] >= 0)[0] # Acciones posibles desde ese estado
    if len(possible_actions) == 0:
        continue # Si no hay acciones válidas, salteamos

    action = random.choice(possible_actions) # Elegimos una acción al azar
    next_possible = np.where(R[action] >= 0)[0] # Acciones posibles desde el siguiente estado

    # Elegimos el máximo valor Q del siguiente estado (para la fórmula de actualización)
    max_q = max(Q[action, next_possible]) if len(next_possible) > 0 else 0

    # Fórmula de actualización Q-learning:
    #  $Q(s, a) = Q(s, a) + \alpha * [R(s, a) + \gamma * \max_{a'}(Q(s', a')) - Q(s, a)]$ 
    Q[state, action] = Q[state, action] + alpha * (R[state, action] + gamma * max_q - Q[state, action])

# Normalizar la matriz Q
# Para hacer más visual la política, normalizamos los valores entre 0 y 1
scaler = MinMaxScaler()
Q_norm = scaler.fit_transform(Q)

# Política óptima: mejor acción desde cada estado
# Para cada estado, elegimos la acción con mayor valor Q
policy = np.argmax(Q, axis=1)

# Creamos un grafo dirigido donde los nodos son las salas y las flechas las transiciones
G = nx.DiGraph()

# Etiquetas legibles para los nodos
labels = {0: "0 (robot)", 1: "1", 2: "2", 3: "3", 4: "4 (tesoro)", 5: "5"}

# Creamos todas las aristas posibles basadas en R (acciones válidas)

```

```

edges = [(i, j) for i in range(n_states) for j in range(n_states) if R[i][j] >= 0]

# Posiciones de Los nodos para una mejor visualización
pos = {
    0: (0, 0),    # robot
    1: (0, 1),
    2: (1, 1),
    3: (1, 0),
    4: (2, 1),    # tesoro
    5: (2, 0)
}

plt.figure(figsize=(8, 6))
nx.draw_networkx_nodes(G, pos, node_size=800, node_color='lightblue')    # Nodos azules
nx.draw_networkx_labels(G, pos, labels, font_size=10)                    # Etiquetas de nodo.

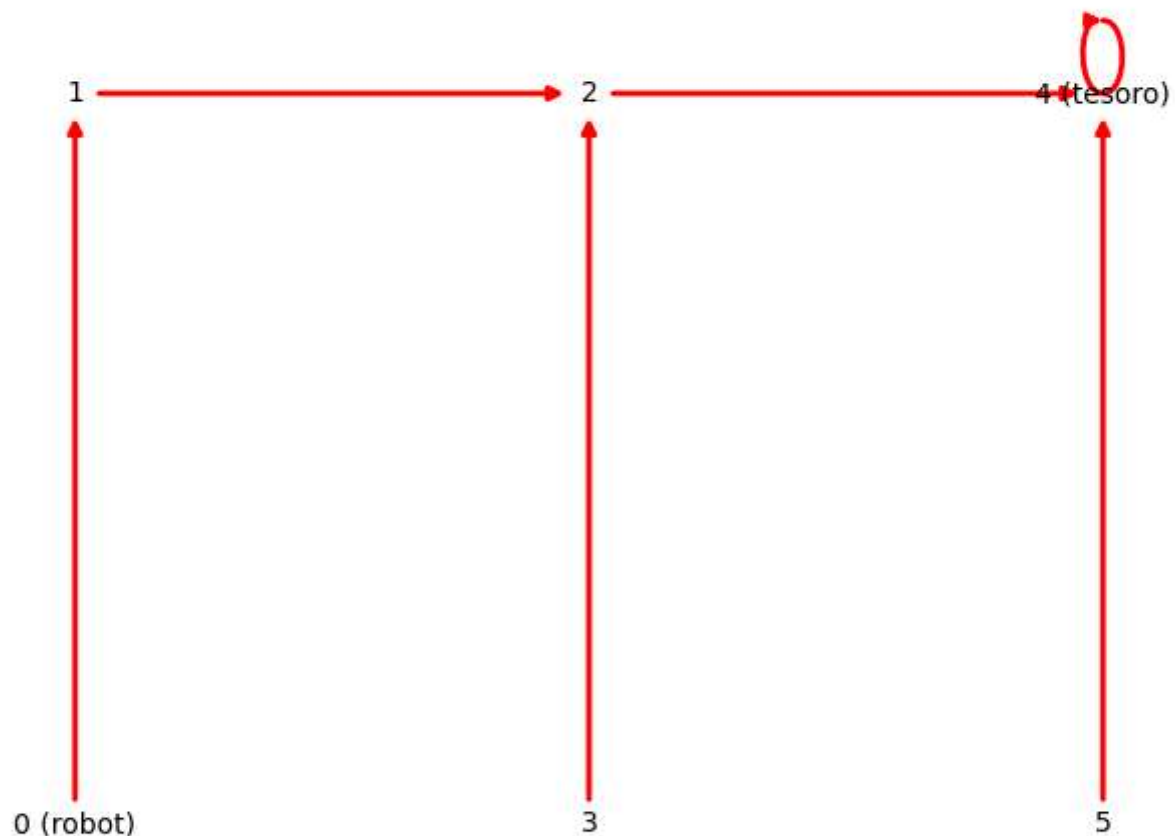
# Dibujamos sólo las transiciones que forman parte de la política óptima
policy_edges = [(s, policy[s]) for s in range(n_states) if R[s][policy[s]] != -1]
nx.draw_networkx_edges(G, pos, edgelist=policy_edges, edge_color='red', width=2, arrows=True)

plt.title("Política óptima aprendida con Q-Learning")
plt.axis('off')
plt.show()

# Mostrar Q normalizada de forma robusta (sin depender de ace_tools)
df = pd.DataFrame(Q_norm, columns=[f"A{i}" for i in range(n_states)])

```

Política óptima aprendida con Q-Learning



Bibliografía

Russell, S. & Norvig, P. (2004) *Inteligencia Artificial: Un Enfoque Moderno*. Pearson Educación S.A. (2a Ed.) Madrid, España

Poole, D. & Mackworth, A. (2017) *Artificial Intelligence: Foundations of Computational Agents*. Cambridge University Press (3a Ed.) Vancouver, Canada